```
-       return <Seat key={seatNumber} seatNumber={seatNumber} />
-     }
-   )
15:   return <tr className="h-20">{seatItems}</tr>

- }
-
- export default Row
```

The basic structure of the component is similar to `Seat`. We have a `props` interface with two fields—one for the row number and one for the number of seats. We have a function named `Row` that returns some JSX that contains an HTML `tr` tag. There are a couple of differences. We need to import the `Seat` component from its file—a component needs to be visible in order to be used.

The biggest oddity, though, is that rather than include each `Seat` inside a loop in the returned JSX, the way we might expect a templating language like ERB to work, we're instead creating all the `Seat` components as an array of components using JavaScript's `map` function. We start with `Array.from(Array(props.seatsPerRow).keys())`, an obscure JavaScript idiom which turns the number `5` into the array `[0, 1, 2, 3, 4]`—it's the closest we can get to a Ruby range object without writing some helper functions. Then we use `map` to create a new array by sending each of those numbers to a function that takes that number and converts it to a React element by returning JSX: `return <Seat key={seatNumber} seatNumber={seatNumber} />`. This code demonstrates an important part of JSX—SX creates regular JavaScript objects that can be returned and managed like any other JavaScript object.

As a result of that `map` call, the `seatItems` variable is set to an array of JSX elements, which we then include in our return value on line 15 using more curly brackets: `return <tr>{seatItems}</tr>`. The array of items are rendered inside the table row and all ends up as we expect.

Why do we need to go through this indirection rather than include a `for` loop inside the `<tr>` tag the way the original ERB did?

The reason is that inside a JSX element, you can only include JavaScript